

PRODUCT REQUIREMENTS DOCUMENT

AI Short-Form Video Clipping Platform

Engineer-ready specification for a Clipworks-style product

Purpose Define the product behavior, functional requirements, technical boundaries, interfaces, quality bars, and phased delivery needed to implement an MVP that turns long-form video into ranked, captioned, vertically reframed short-form clips.

Version 1.0 | September 2026

Primary implementation stack: Next.js, NestJS, PostgreSQL, Redis/BullMQ, Python, FFmpeg, Cloudflare R2

Target initial deployment: single 8 GB Hetzner server, with external AI APIs and controlled render concurrency

CONFIDENTIAL

1. Document Control

Field	Value
Document	Product Requirements - AI Short-Form Video Clipping Platform
Version	1.0
Status	Implementation baseline
Audience	Product engineer, backend engineer, ML/media engineer, frontend engineer, QA
Product type	Web SaaS
Primary use case	Turn long-form creator video into ranked short-form clips for TikTok, Instagram Reels, and YouTube Shorts
MVP hosting	8 GB Hetzner server + Cloudflare R2 + hosted AI APIs
Out of scope for MVP	Full non-linear editor, social publishing, proprietary model training, team collaboration, mobile apps

1.1 Definitions

Term	Definition
Source video	The long-form input video submitted by a user.
Candidate	A timestamp range identified by the analysis pipeline as potentially suitable for a short-form clip.
Rendered clip	An encoded output video produced from a candidate, including layout/reframing and optional burned-in captions.
Transcript segment	A timestamped utterance or sentence-level transcription unit.
Word timing	Start/end timestamps for individual spoken words.
Virality score	A product ranking score from 0-100 estimating short-form suitability; it is not a prediction guarantee.
Smart reframe	Automatic transformation from landscape or arbitrary source framing into a target aspect ratio while keeping important subjects visible.
Job	A durable asynchronous unit of work processed through a queue.
Provider	A swappable external or local implementation for transcription, language-model inference, or other AI services.

2. Product Summary

The product accepts a long-form video, transcribes and analyzes it, identifies self-contained moments with short-form potential, ranks those moments, and lets the user generate finished vertical clips with smart reframing and animated captions. The system is designed to minimize unnecessary rendering: expensive visual analysis and encoding happen only for clips the user chooses to generate.

2.1 Product promise

Core promise A creator should be able to submit a long video and receive a useful, ranked shortlist of short-form moments without manually scrubbing the entire recording. From that shortlist, the creator can generate a polished 9:16 clip with captions in a few clicks.

2.2 Primary outcomes

- Reduce the time required to find short-form moments in long videos.
- Generate clips that begin and end at natural speech boundaries.
- Keep the active or most relevant subject visible in vertical framing.
- Produce readable, well-timed short-form captions from word-level timestamps.
- Keep MVP infrastructure simple enough to operate on one modest server.
- Make AI/transcription providers replaceable without rewriting the pipeline.

2.3 Non-goals for MVP

- A Premiere Pro/CapCut-class editor.
- Automatic publishing to TikTok, Instagram, or YouTube.
- Training a proprietary transcription, ranking, or computer-vision model.
- Guaranteed viral-performance prediction.
- Live-stream clipping in real time.
- Multi-user workspaces, review comments, or enterprise permissions.
- GPU orchestration, Kubernetes, or autoscaling infrastructure.

3. Users, Jobs and Core User Journey

3.1 Primary user

The MVP is designed for an individual creator, editor, podcaster, streamer, educator, interviewer, or small content team member who owns or has authorization to process the source material and wants to repurpose long-form content into short-form social clips.

3.2 Core jobs to be done

User job	Desired outcome
Submit long-form content	Provide a file or supported source URL without uploading through the application server.
Find the best moments	See ranked candidate clips with timestamps, transcript excerpts, titles, scores, and reasons.
Generate a clip	Create a 9:16 output with subject-aware framing and captions.
Adjust a result	Modify start/end times, caption preset, and framing preference, then re-render.
Retrieve output	Preview and download the completed MP4 via signed storage URL.
Understand failures	See a clear state and retryable/non-retryable error when processing fails.

3.3 Golden-path journey

1. User signs in and selects New Video.
2. User chooses direct upload or pastes a supported source URL.
3. For direct upload, the frontend uploads the source directly to R2 using a signed multipart-capable upload flow.
4. The API creates the video record and enqueues ingestion.
5. The worker validates media, extracts metadata and audio, and initiates transcription.
6. The transcript is normalized into timestamped segments and words.
7. The analysis service chunks the transcript, generates candidate moments, deduplicates overlaps, refines boundaries, and ranks candidates.
8. The user sees a candidate list ordered by score. The source player can seek to each candidate range.
9. The user selects a candidate and clicks Generate Clip.
10. The render pipeline analyzes faces/subjects only inside that candidate window, determines a crop/layout, generates caption timing/style, and renders the final MP4.
11. The completed clip is uploaded to R2 and becomes previewable/downloadable.
12. The user may adjust boundaries/style and create a new render without repeating transcription or global analysis.

3.4 User-visible processing states

Stage	User-facing label	Behavior
UPLOADING	Uploading video	Frontend-managed progress for direct upload.
INGESTING	Preparing video	Validate file, metadata and source availability.
TRANSCRIBING	Creating transcript	Audio is being transcribed and aligned.

ANALYZING	Finding best moments	Transcript is chunked, candidates generated and ranked.
READY	Clips found	Candidate list available; no final render is required yet.
RENDERING	Generating clip	Selected candidate is being reframed, captioned and encoded.
COMPLETED	Clip ready	Rendered clip is available.
FAILED	Processing failed	Show actionable error and retry when safe.

4. Scope and Priority

4.1 MVP - P0 requirements

Area	P0 requirement
Authentication	Email/password or a single managed auth provider; authenticated user owns all videos/clips.
Input	Direct upload to R2. YouTube URL ingestion supported through a replaceable source adapter where permitted.
Media validation	MP4/MOV/WebM input, configurable max duration and max file size, ffprobe metadata extraction.
Transcription	Hosted transcription provider by default; normalized word-level timestamps; provider abstraction required.
Analysis	Transcript chunking, LLM candidate generation, overlap deduplication, boundary refinement, scoring and ranking.
Candidate UI	Candidate list with title, timestamp range, score, reason, transcript excerpt, and seek-to-source preview.
Rendering	On-demand 9:16 MP4 render, H.264/AAC, face/subject-aware crop, burned-in captions.
Editing	Adjust clip start/end, choose caption preset, choose auto/center crop, rerender.
Storage	Cloudflare R2 for source and outputs; signed URLs; deterministic object keys; cleanup policy.
Jobs	Redis/BullMQ queues, retries, idempotency, progress and error persistence.
Deployment	Dockerized services suitable for one 8 GB Hetzner host, with render concurrency = 1 initially.
Observability	Structured logs, job duration metrics, error codes, disk usage guardrails.

4.2 Post-MVP - P1/P2

Priority	Feature
P1	Twitch/VOD source adapters; additional import providers.
P1	Multiple caption themes and per-word emphasis controls.
P1	Advanced active-speaker detection using audio/lip synchronization.
P1	Split-screen and gameplay+speaker layout strategies.
P1	Auto-render top N candidates as a configurable option.
P1	Billing/credits, usage limits and subscription plans.
P1	Clip title/description/hashtag generation.

AI SHORT-FORM VIDEO CLIPPING PLATFORM / PRODUCT REQUIREMENTS

P2	Social publishing and account connections.
P2	Full timeline editor and manual keyframes.
P2	Team workspaces, comments and review workflows.
P2	GPU worker pool or managed render/transcription cluster.
P2	Learning-to-rank system informed by user selections and clip performance.

5. Functional Requirements - Video Ingestion

5.1 Create video

The API must create a durable video record before any long-running processing begins. A video ID becomes the correlation identifier used by storage, queues, logs, progress events and child clips.

FR-ING-001 Direct upload

- The frontend requests an upload session from NestJS.
- NestJS returns signed R2 upload information. The file payload must not transit through NestJS.
- Use multipart upload for large files so the system is not constrained by a single-request object-size limit.
- The client reports completion to the API with object key, original filename, MIME type and observed size.
- The API validates that the object exists before enqueueing ingestion.

FR-ING-002 URL ingestion

- A SourceAdapter interface must encapsulate provider-specific URL download behavior.
- The YouTube adapter is P0 only where the user is authorized to process the video and implementation is legally/operationally permitted.
- Provider-specific extraction logic must not be embedded in core job code.
- Unsupported, private, age-gated or inaccessible sources must fail with a non-retryable user-facing error.

FR-ING-003 Validation

Check	Requirement
Container/codec	Use ffprobe. Accept formats FFmpeg can reliably decode; reject unreadable/corrupt media.
Duration	Persist duration in milliseconds. Enforce a configurable plan/system maximum.
Dimensions	Persist width, height, orientation and frame rate.
Audio	Detect whether an audio stream exists. Candidate discovery based on speech requires audio.
Size	Persist byte size; enforce configurable maximum.
Rights confirmation	UI must require the user to confirm they own or are authorized to process submitted content.

Acceptance criteria

- A valid direct upload creates one video record and exactly one ingest job.
- Refreshing the page does not duplicate ingestion.
- An already-completed upload callback can be safely retried.
- Corrupt or unsupported files transition to FAILED with a stable error code.
- Temporary local files are removed after success and after terminal failure.

6. Functional Requirements - Transcription

6.1 Provider abstraction

Transcription must be implemented behind an interface so the application can switch between a hosted API and local WhisperX without changing downstream analysis.

```

TranscriptionProvider
  transcribe(input: AudioInput) -> TranscriptResult
  get_status?(providerJobId) -> TranscriptResult | Pending

```

Implementations:

```

HostedTranscriptionProvider [P0 default]
WhisperXProvider           [optional/local adapter]

```

6.2 Audio preparation

- Extract a mono speech-optimized audio asset from the source with FFmpeg.
- Prefer a compressed format accepted by the provider to reduce upload time and local disk usage.
- Do not create uncompressed WAV unless required by a provider or alignment model.
- Store intermediate audio locally only for the duration of the transcription job unless debugging retention is explicitly enabled.

6.3 Normalized transcript schema

```

TranscriptResult {
  language: string
  durationMs: number
  fullText: string
  segments: [{ id, startMs, endMs, text, speaker? }]
  words: [{ startMs, endMs, text, confidence?, speaker? }]
  provider: string
  providerMetadata?: object
}

```

6.4 Requirements

- Word-level timings are required for caption rendering.
- Sentence/segment timings are required for boundary refinement and candidate preview.
- The transcript must be persisted before global highlight analysis begins.
- Provider-specific JSON must not leak into downstream business logic.
- If the provider is asynchronous, the API/worker must persist provider job ID and resume without holding an HTTP connection open.
- A local WhisperX adapter may use CPU/int8 for low-volume deployment, but it must be disabled by default on the same host when it would compete with render workloads.

Acceptance criteria

- A 60+ minute source can be transcribed without holding a frontend or API request open.
- The persisted transcript can be re-used to regenerate candidates without retranscribing.
- At least 99% of transcript words with spoken content have monotonic, non-negative timing values.
- Provider failure is retryable where appropriate and does not create duplicate transcript records.

7. Functional Requirements - Highlight Discovery

7.1 Analysis strategy

The system must not send an entire long-form transcript to one unconstrained prompt. It must chunk the transcript into bounded, overlapping windows, generate local candidates, normalize them to source time, deduplicate overlaps, refine boundaries, then run a second-pass ranking step.

7.2 Default chunking

Parameter	Default	Configurable
Window length	5 minutes	Yes
Overlap	1 minute	Yes
Candidate target length	20-90 seconds	Yes
Candidates per window	0-5	Yes
Global shortlist	Up to 15	Yes

7.3 Candidate generation criteria

The LLM should favor moments that are understandable with limited external context and contain at least one strong short-form signal.

- Strong hook or curiosity gap near the beginning.
- Surprising, contrarian or specific claim.
- Clear advice, explanation, list, story payoff or lesson.
- Emotional intensity, humor, tension or memorable reaction described in speech.
- High information density with low filler.
- A natural ending or conclusion inside the target duration.
- Minimal dependency on statements outside the clip.

7.4 Candidate data contract

```
CandidateProposal {
  sourceStartMs: number
  sourceEndMs: number
  hookText: string
  summary: string
  reason: string
  category: "insight" | "story" | "humor" | "controversy" | "advice" | "reaction" | "other"
  modelScores: { hook, clarity, novelty, emotion, completeness, shareability } // 0-10
}
```

7.5 Deduplication

- Compute temporal intersection-over-union between candidate ranges.
- Candidates with IoU above the configured threshold (default 0.5) are treated as duplicates unless their semantic content is materially distinct.
- When duplicates collide, keep the stronger-ranked candidate and optionally expand its boundaries if doing so improves completeness.
- Optional P1 semantic deduplication may compare embeddings of transcript excerpts.

7.6 Boundary refinement

- Snap the candidate start to the beginning of a nearby sentence/segment whenever possible.
- Avoid starting with a dangling conjunction, pronoun without referent, filler-only phrase, or the end of a previous thought.
- Prefer ending after a complete sentence, punchline, answer or payoff.
- Allow a small configurable pre/post roll of approximately 200-500 ms after alignment.
- Hard-limit final candidate duration to configured minimum/maximum unless an administrator overrides it.

7.7 Ranking and virality score

The product score is a deterministic aggregation of model-assessed dimensions, not a guaranteed prediction. Persist both the aggregate and component values.

Dimension	Default weight	Meaning
Hook strength	25%	How quickly the moment earns attention.
Standalone clarity	20%	How understandable it is without surrounding context.
Novelty/specificity	15%	How non-generic or surprising the content is.
Emotional intensity	15%	Humor, tension, surprise, excitement, empathy.
Completeness/payoff	15%	Whether the clip resolves its premise.
Shareability	10%	Likelihood someone would quote, save or send it.

Acceptance criteria

- Analysis returns zero candidates rather than inventing weak moments when the source has no suitable content.
- Candidate timestamps always fall within source duration.
- The global list contains no obvious temporal duplicates after deduplication.
- Each candidate has a title/hook, excerpt, reason, component scores and aggregate score.
- Re-running analysis with the same pipeline version can replace candidates without modifying already-rendered clip records.

8. Functional Requirements - Candidate Experience

8.1 Candidate list

The candidate page is the main result of the analysis phase. It must be useful before any expensive clip rendering occurs.

Each candidate card must show

- Generated short title.
- Start and end timestamp plus duration.
- Virality score from 0-100.
- One-sentence explanation of why the moment was selected.
- Transcript excerpt.

- Category/tag.
- Play/preview action that seeks the original source player to the candidate range.
- Generate Clip action.

8.2 Sorting/filtering

- Default sort: score descending.
- Allow sort by source time.
- P1: filter by category or duration.

8.3 No-candidate state

If no candidate meets the minimum quality threshold, show a neutral result explaining that no strong short-form moments were detected and allow the user to lower the threshold or create a manual range. Do not fabricate high scores.

9. Functional Requirements - Rendering and Smart Reframing

9.1 Render initiation

A render is a separate durable entity from the candidate. Clicking Generate Clip creates a render job using an immutable snapshot of the candidate range and render settings. Rerendering creates a new render attempt/version rather than mutating a completed file in place.

9.2 Default output specification

Property	MVP default
Aspect ratio	9:16
Resolution	1080 x 1920
Video codec	H.264
Audio codec	AAC
Container	MP4
Frame rate	Preserve source when practical; otherwise configured standard
Audio	Preserve source mix; normalize only if configured
Subtitles	Burned-in ASS captions
Metadata	Strip unnecessary source metadata from final output

9.3 Clip extraction

- Use source-accurate timestamps from the refined candidate.
- For final output, correctness takes precedence over fastest seek/copy behavior.
- If encoding is required for crop/captions, perform one final video encode rather than stacking multiple lossy transcodes.
- Keep source audio synchronized after clipping and rendering.

9.4 Face/subject analysis

Visual analysis must be limited to the selected candidate window, not the full source video. The MVP may use CPU-friendly face detection and tracking.

- Sample frames at a configurable analysis rate, initially 2-5 FPS.
- Detect faces with MediaPipe, InsightFace, a lightweight YOLO model, or an equivalent CPU-capable detector.
- Track detected subjects between samples to reduce jitter and preserve identity.
- Store analysis coordinates in source-frame pixel or normalized coordinates with explicit frame dimensions.
- Interpolate crop positions for frames between analysis samples.

9.5 Speaker/subject selection - MVP heuristic

- If one stable face dominates the clip, center framing on that face with headroom.
- If multiple faces are present, prefer the face most consistently visible and closest to the source center unless speaker information can be inferred reliably.

- If diarized speaker labels and face tracks can be confidently associated, follow the active speaker.
- If confidence is low, fall back to a center crop or fit-with-background strategy rather than making rapid incorrect jumps.
- Advanced audio-visual active speaker detection is P1.

9.6 Crop smoothing

Raw face coordinates must never be used directly as per-frame crop coordinates. Apply an exponential moving average, Kalman filter, spline/interpolation strategy, or equivalent smoothing with velocity limits so the virtual camera moves naturally.

9.7 Safe framing rules

- Keep the tracked face fully inside a configurable safe region.
- Avoid cropping through eyes/chin where possible.
- Prevent crop coordinates from exceeding source frame bounds.
- Apply a minimum dwell time before switching between subjects.
- Preserve enough lower-frame room for captions when possible.

Acceptance criteria

- Final MP4 plays in standard mobile/web players and is 9:16 at the configured resolution.
- No frame has an out-of-bounds crop or black edge caused by invalid crop coordinates.
- The primary visible face remains in frame for the substantial majority of applicable frames.
- Camera movement is visibly smoothed; minor detector jitter must not translate into frame-to-frame shaking.
- If face detection fails, rendering still completes using the configured fallback layout.

10. Functional Requirements - Captions

10.1 Caption source

Captions must be generated from normalized word timings, not re-transcribed from the clip. Candidate-local timestamps are derived by subtracting the rendered clip start from source-level timings.

10.2 Grouping

- Group words into short, readable phrases based on timing gaps, punctuation and maximum character/word count.
- Prefer 1-2 visual lines.
- Avoid leaving a single short word on a line when regrouping can improve readability.
- Caption phrases must never display before their first word is spoken or remain long after the final word.

10.3 Styling

- MVP includes at least one polished preset with high-contrast text, outline/shadow, bottom-safe positioning and active-word emphasis.
- ASS subtitle generation is preferred for styling and per-word timing.
- Caption layout must stay inside mobile-safe margins.
- The user can toggle captions on/off and choose from the MVP preset(s).

10.4 Transcript cleanup

Caption cleanup may remove obvious filler repetitions or normalize punctuation/casing only when it does not change meaning. The displayed captions should remain faithful to the spoken content. Do not invent words or paraphrase claims in a way that changes what the speaker said.

Acceptance criteria

- Caption timing follows speech closely and remains synchronized throughout the clip.
- No caption is clipped by the output frame.
- Current-word emphasis advances monotonically with word timings.
- Turning captions off yields a valid render with the same crop and audio.

11. Clip Adjustment and Rerendering

11.1 Editable settings

Setting	MVP behavior
Start/end	User can change within the source video; enforce minimum/maximum duration.
Aspect ratio	9:16 only in P0; 1:1 and 16:9 are P1.
Framing	Auto smart crop or center crop.
Captions	On/off plus preset selection.
Caption position	Preset-driven in P0; manual vertical offset P1.
Title	Editable metadata; does not burn into video in P0.

11.2 Rerender semantics

- A rerender never repeats transcription or global candidate discovery.
- If the timestamp window changes, face analysis and caption subset extraction are recomputed only for the new range.
- A render version records the complete settings snapshot used to create it.
- Previous successful renders may be retained or cleaned according to retention policy; the database must distinguish the active/latest render.

12. API Requirements

Exact route names may change, but the capabilities and idempotency semantics below are required. All endpoints require authentication unless explicitly public.

Method / path	Purpose	Notes
POST /videos	Create video record	Input type: upload or URL.
POST /videos/:id/upload-session	Create signed R2 upload session	Supports multipart where needed.
POST /videos/:id/upload-complete	Confirm object upload	Idempotent; enqueue ingestion once.
POST /videos/:id/process	Start/restart process	Only when state allows.
GET /videos/:id	Video detail + progress	Return durable status, metadata and errors.
GET /videos/:id/candidates	Ranked candidates	Pagination optional for MVP.
POST /videos/:id/analyze	Re-run candidate analysis	Uses existing transcript.
POST /candidates/:id/renders	Create render	Settings snapshot in body.
GET /renders/:id	Render status + output	Return signed playback URL when ready.
POST /renders/:id/retry	Retry failed render	Idempotent policy required.
DELETE /videos/:id	Delete user video	Triggers storage cleanup according to policy.

12.1 Example create-video payload

```
{
  "sourceType": "upload",
  "originalFilename": "podcast-episode.mp4",
  "contentType": "video/mp4",
```

```
{
  "sizeBytes": 4318827341
}
```

12.2 Example create-render payload

```
{
  "startMs": 842310,
  "endMs": 891640,
  "aspectRatio": "9:16",
  "framingMode": "auto",
  "captions": { "enabled": true, "preset": "bold-default" }
}
```

12.3 Error response

```
{
  "error": {
    "code": "VIDEO_UNSUPPORTED_CODEC",
    "message": "This video format could not be decoded.",
    "retryable": false,
    "correlationId": "..."
  }
}
```

13. Data Model

The engineer may adapt ORM details, but the following conceptual entities and relationships must exist. Use UUID/ULID-style IDs and UTC timestamps. Store duration/timing in integer milliseconds to avoid floating-point drift.

Entity	Required fields / purpose
users	id, auth identity, createdAt, plan/limits placeholder.
videos	id, userId, sourceType, sourceUrl/objectKey, filename, MIME, size, durationMs, width, height, fps, language, status, progress, pipelineVersion, errorCode, createdAt, updatedAt.
transcripts	id, videoId, provider, language, fullText, segments JSONB, words JSONB, providerJobId, status, version, createdAt.
candidates	id, videoId, startMs, endMs, title, hookText, excerpt, summary, reason, category, score, componentScores JSONB, analysisVersion, rank, createdAt.
renders	id, candidateId nullable, videoId, userId, startMs, endMs, settings JSONB, status, progress, outputObjectKey, width, height, durationMs, fileSize, errorCode, version, createdAt, completedAt.
jobs / job_runs	Optional DB mirror of important queue jobs: queue name, external job id, entity id, attempt, status, startedAt, completedAt, error metadata.
usage_events	P1 billing-ready ledger: userId, event type, video/render id, units, idempotency key, createdAt.

13.1 Important constraints

- One active canonical transcript version per video.
- Unique idempotency key for upload completion and billable usage events.
- Candidate startMs < endMs and both within video duration.

- Render startMs < endMs and both within video duration.
- All object keys are owned by a user/video prefix and are never accepted blindly from arbitrary client input.
- Deleting a video must cascade/soft-delete related candidates/renderers and schedule storage cleanup.

14. Asynchronous Job Architecture

14.1 Queues

Queue	Work	Initial concurrency on 8 GB host
video-ingest	Download/validate source, ffprobe, audio extraction	1-2
transcription	Send/poll hosted transcription provider or run local adapter	2-4 for hosted API; 1 for local WhisperX
analysis	LLM candidate generation/ranking	4-10 because mostly network-bound
render	Face analysis, captions, FFmpeg encoding, R2 upload	1
cleanup	Remove temp/storage artifacts and expired outputs	1

14.2 Orchestration

```

UPLOAD/URL
|
v
video-ingest
| - validate + ffprobe
| - extract audio
v
transcription
| - normalize + persist transcript
v
analysis
| - chunk -> generate -> dedupe -> refine -> rank
v
VIDEO READY / candidate list

User selects candidate
v
render
| - clip-local face analysis
| - crop path + captions
| - FFmpeg encode
| - upload output
v
RENDER COMPLETED

```

14.3 Job payload principles

- Queue payloads carry IDs and immutable parameters, not large transcript/video blobs.
- Workers fetch durable state from PostgreSQL/R2.
- Each job has a deterministic idempotency key derived from operation + entity + version where feasible.
- Retries must be safe. Side effects such as creating transcripts, outputs or usage charges must use upsert/idempotent semantics.

- A worker crash must not require repeating already-completed prior stages.

14.4 Retry policy

Failure class	Default policy
Transient network/provider 5xx	Exponential backoff, 3-5 attempts.
Rate limit	Retry respecting provider retry-after/backoff.
Corrupt/unsupported media	No retry; user-visible terminal error.
FFmpeg process crash/OOM	One controlled retry after cleanup; then fail and record diagnostic.
Storage timeout	Retry upload/download with bounded attempts.
Invalid LLM structured output	One or more repair/re-prompt attempts; fail analysis stage if schema remains invalid.
Worker killed/restarted	BullMQ reclaims/stalled-job handling; all stages remain idempotent.

15. Progress and Realtime Updates

Progress is advisory; durable status in PostgreSQL is authoritative. The frontend may receive realtime updates through SSE or WebSocket, with polling fallback.

Stage	Suggested progress range
Ingest	0-15%
Transcription	15-50%
Global analysis	50-85%
Candidate finalization	85-100%
Render - separate progress	0-100% within render entity

- Workers update stage, percent and a short machine-readable substage.
- Do not imply precise time remaining unless measured from real historical job data.
- A reconnecting client can reconstruct state from GET endpoints without depending on missed realtime events.

16. Storage and File Lifecycle

16.1 R2 object layout

```
users/{userId}/videos/{videoId}/source/{originalName}
users/{userId}/videos/{videoId}/derived/audio.{ext} [optional/temporary]
users/{userId}/videos/{videoId}/renders/{renderId}/final.mp4
users/{userId}/videos/{videoId}/renders/{renderId}/thumb.jpg
```

16.2 Local working directory

```
/tmp/video-jobs/{jobId}/
source.*
audio.*
clip-input.*
captions.ass
output.mp4
frames/ [only if required]
```

16.3 Lifecycle rules

- Each job owns an isolated working directory.
- Always delete the working directory in a finally/cleanup path after success or terminal failure.
- Run a periodic janitor that removes abandoned working directories older than a safety threshold.
- Monitor free disk before starting a render; fail or defer gracefully if the safety reserve would be breached.
- Use signed, time-limited URLs for upload and playback/download.
- Never expose R2 master credentials to the browser.
- Source and output retention must be configurable. User deletion must schedule corresponding storage deletion.

17. Infrastructure and Deployment

17.1 Initial single-server topology

```

8 GB Hetzner host
Docker Compose
- reverse proxy / TLS
- Next.js web
- NestJS API
- PostgreSQL
- Redis
- Python worker
  * BullMQ consumer
  * FFmpeg / ffprobe
  * OpenCV / face detection

External
- Cloudflare R2
- hosted transcription API
- LLM API

```

17.2 Initial resource policy

- Render queue concurrency = 1.
- Do not run a large local language model on the same 8 GB production host.
- Hosted transcription is the default to preserve CPU for FFmpeg.
- If local WhisperX is enabled, set transcription concurrency = 1 and prevent it from overlapping with render jobs unless resource measurements show sufficient headroom.
- Set Docker memory/CPU limits where useful so one worker cannot starve PostgreSQL/API.
- Use a bounded temp directory and disk free-space guard.

17.3 Scale-out path

The architecture must allow the worker to move off the API/database host without application redesign. Redis and PostgreSQL remain the coordination layer; additional worker machines consume the same queues.

```

Host A: Web + NestJS + Postgres + Redis
Host B: Python worker / render
Host C: Python worker / render
Host D: GPU worker (optional later)

```

17.4 Environment configuration

Category	Examples
----------	----------

18. Security, Privacy and Abuse Controls

- Every video, candidate and render query must be scoped to the authenticated user.
- Use signed upload/download URLs with short expirations.
- Validate MIME/container with server-side probing; do not trust filename extension.
- Treat source URLs as untrusted input. Defend against SSRF: provider adapters must resolve only supported hosts or use a hardened downloader policy.
- Run FFmpeg and media tools with least privilege and no unnecessary host mounts.
- Set request/body limits on API endpoints; file upload occurs directly to object storage.
- Sanitize generated filenames and never shell-concatenate user input into FFmpeg/yt-dlp commands. Use subprocess argument arrays.
- Secrets must come from environment/secret storage, never source control.
- Provide a deletion path for user content and derived files.
- Require users to confirm they have rights/authorization to process submitted media.
- Apply rate limits to video creation, analysis and render endpoints to prevent resource exhaustion.

19. Observability and Operations

19.1 Structured logging

Every log related to a pipeline operation should include correlation identifiers where applicable: `userId`, `videoId`, `candidateId`, `renderId`, `queueName`, `jobId`, `attempt` and `pipelineVersion`. Do not log access tokens, provider secrets or full signed URLs.

19.2 Metrics

Metric	Why it matters
Jobs completed/failed by queue	Reliability.
Stage duration histograms	Bottleneck detection and later ETA estimation.
Render duration per output minute	Capacity planning.
Transcription provider latency/cost units	Provider economics.
LLM tokens/calls by analysis	Cost control.
Temp disk high-water mark	Prevent disk exhaustion.
Queue depth/wait time	When to add workers.
Candidate count and render-selection rate	Product quality signal.
Rerender rate	Indicates poor default crop/caption/boundary quality.

19.3 Error taxonomy

VIDEO_*	source/media problems
UPLOAD_*	storage/upload problems
TRANSCRIPTION_*	provider/audio/alignment problems
ANALYSIS_*	LLM/schema/candidate problems
RENDER_*	face/crop/caption/FFmpeg problems
STORAGE_*	R2 transfer/signing problems
SYSTEM_*	resource/configuration/internal problems

20. Non-Functional Requirements

Area	Requirement
Reliability	Long-running work must survive HTTP disconnects and worker restarts. Durable state must allow stage-level retries.
API latency	Normal metadata endpoints should target sub-second p95 under MVP load, excluding provider and upload operations.
Scalability	Worker processes must be horizontally replicable without code changes to the API contract.
Memory	A single render job must fit within the 8 GB host with the rest of the stack running; concurrency must be configured accordingly.
Disk	Pipeline must actively manage temporary assets and preserve a free-space reserve.
Idempotency	Repeating upload-complete, process, provider callback or retry operations must not duplicate durable side effects.
Data integrity	All timestamps use integer milliseconds; all status transitions are validated; completed output metadata matches actual object/file.
Compatibility	Final MP4 must play in modern browsers and common mobile/social workflows.
Maintainability	Provider/source/render logic is modular, typed where practical, and covered by integration tests around contracts.
Cost awareness	Do not run full-video visual analysis or render every candidate by default.
Accessibility	Core web UI must support keyboard navigation, readable contrast and semantic status text in addition to visual progress.

21. Frontend Product Requirements

21.1 Required screens

Screen	Requirements
Sign in	Minimal authentication flow.
Dashboard	List videos with thumbnail, title/filename, duration, status, created date; New Video CTA.
New Video	Upload dropzone/file picker, URL input, rights confirmation, file requirements.
Processing	Source summary, stage/progress, cancel/delete where safe, automatic transition when ready.
Video results	Source player + ranked candidate list; click candidate seeks to range; Generate action.
Render detail	Preview output, status, download, start/end controls, framing mode, captions toggle/preset, rerender.
Failure state	Human-readable error, correlation/reference ID, retry when allowed.

21.2 UI behavior

- Never block the browser on a long-running synchronous request.

- Persist navigation-safe IDs in the URL so refresh/reopen works.
- Show deterministic statuses from backend state, not optimistic assumptions.
- Disable Generate while an identical render settings snapshot is already processing unless duplicate renders are intentionally allowed.
- Preview candidate ranges using the original source asset where browser-compatible; generate a proxy source only if necessary.
- Signed media URLs should be refreshed transparently when expired.

22. Quality and Acceptance Test Matrix

Scenario	Expected result
Valid 60-minute MP4 upload	Uploads directly to R2, ingests, transcribes, analyzes, and shows candidates without API timeout.
4+ GB source	Multipart upload path works; NestJS does not buffer the file.
Corrupt video	FAILED with non-retryable media error; temp files removed.
Video with no audio	Clear unsupported/no-speech behavior; no meaningless LLM analysis.
No strong highlights	READY with empty/low-confidence result state, not fabricated high scores.
Overlapping LLM proposals	Deduplication leaves one stronger canonical candidate.
User refreshes during transcription	Page reconstructs current status from API and continues updating.
Worker restarts mid-job	Job is retried/resumed safely; no duplicate transcript/render charge.
One-person landscape clip	9:16 output keeps face visible with stable crop.
Two-person clip, uncertain speaker	Fallback framing avoids rapid incorrect jumps.
Face detector finds nothing	Center/fallback render still completes.
Captioned render	Words remain synchronized and inside safe bounds.
Captions disabled	Output renders without subtitles.
Boundary edit + rerender	No retranscription/global reanalysis; only range-local render work repeats.
R2 temporary URL expired	Application refreshes/signs a new URL without changing object identity.
Delete video	Database state changes and all owned storage artifacts are scheduled for deletion.
Concurrent render requests	Queue serializes render work at configured concurrency 1 on initial host.
Low disk reserve	New heavy work is deferred/failed gracefully before disk exhaustion.

23. Testing Requirements

23.1 Unit tests

- Candidate temporal IoU and deduplication.
- Boundary snapping/refinement against transcript segments.
- Virality score aggregation and weighting.
- Caption grouping and source-to-clip timestamp conversion.
- Crop boundary clamping and smoothing math.
- State transition validation.
- Idempotency key generation.
- Storage object key construction and ownership checks.

23.2 Integration tests

- NestJS -> Redis/BullMQ -> Python worker contract.
- R2 signed upload and object verification.
- Transcription provider adapter using a fixture/mock provider.
- LLM structured-output schema validation and repair path.
- FFmpeg render of controlled fixtures: landscape single speaker, two speakers, no face, portrait input, variable frame rate where supported.
- Worker crash/retry and temp cleanup.
- Deletion and storage cleanup.

23.3 Golden media fixtures

Maintain a small internal fixture library with known expected behavior. Tests do not need pixel-perfect crop equality, but should assert valid video output, duration bounds, caption presence, no out-of-frame crop coordinates, and stable subject tracking metrics where measurable.

24. Implementation Plan

Phase	Deliverable	Exit criteria
1. Foundation	Monorepo/repositories, Docker Compose, NestJS, Postgres, Redis, Python BullMQ worker, config, auth skeleton.	API and worker communicate through a test job; services boot on target host.
2. Ingestion	Video records, R2 direct upload, source adapters, fprobe, temp workspace.	Uploaded source can be validated and metadata persisted.
3. Transcription	Audio extraction, provider abstraction, hosted provider, normalized transcript persistence.	Long video produces reusable word-timed transcript.
4. Highlight engine	Chunking, LLM structured candidates, dedupe, boundary refinement, ranking.	Candidate list is useful without rendering.
5. Candidate UI	Dashboard, processing page, source player and candidate cards.	User can submit and review candidates end-to-end.
6. Render MVP	Clip extraction, center crop fallback, ASS captions, FFmpeg encode, R2 output.	Selected candidate generates a valid downloadable 9:16 MP4.
7. Smart reframe	Face detection/tracking, crop smoothing, fallback modes.	Single- and multi-face fixtures produce stable acceptable framing.
8. Rerender/editor	Boundary controls, caption/framing settings, render versions.	User can adjust and rerender without repeating upstream stages.

9. Hardening	Retries, idempotency, metrics, rate limits, cleanup, security review.	Acceptance matrix passes on staging and target 8 GB host.
10. P1 readiness	Billing hooks, additional presets/providers, worker scale-out docs.	Architecture supports next features without rewrites.

25. Definition of Done - MVP

The MVP is considered implementation-complete when all of the following are true:

- An authenticated user can submit a supported direct-upload video and start processing.
- The source is stored in R2 and not proxied through the NestJS process.
- The pipeline creates a word-timed transcript using the configured transcription provider.
- The system returns a deduplicated, ranked list of candidate short-form moments with natural boundaries and explanations.
- The candidate list is usable before final clips are rendered.
- The user can generate a selected candidate into a 1080x1920 H.264/AAC MP4.
- The render supports stable subject-aware framing with a reliable fallback.
- The render supports burned-in, synchronized captions generated from transcript word timings.
- The user can modify start/end, framing mode and caption setting and rerender without retranscription.
- All long-running stages are queued and survive browser disconnects.
- Retries are idempotent and do not create duplicate durable side effects.
- Temporary files are cleaned up after success/failure and the host is protected against disk exhaustion.
- The system runs reliably on the initial 8 GB Hetzner deployment with render concurrency set to one.
- Core security requirements, logging, metrics and acceptance tests are implemented.
- Deployment and environment configuration are documented sufficiently for another engineer to reproduce staging/production.

26. Engineering Decisions and Constraints

Decision	Rationale
NestJS for product API	Strong fit for auth, database, API contracts, job creation and product logic.
Python for media/ML worker	Best ecosystem fit for CV, WhisperX, OpenCV and media analysis.
BullMQ + Redis	Simple durable queue already compatible with Node and Python worker patterns; avoids adding RabbitMQ for MVP.
PostgreSQL	Durable product state, JSONB transcript/score storage and reliable transactional semantics.
R2 object storage	Cheap scalable source/output storage and direct signed uploads.
FFmpeg/ffprobe	Media probing, extraction, crop, subtitle burn-in and final encoding.
Hosted transcription first	Preserves CPU/RAM on the 8 GB host and reduces processing contention.
WhisperX behind adapter	Enables local/cheaper transcription later without coupling downstream logic.
LLM API for highlight analysis	No need to host a large model on the MVP server.
Analyze visuals only for selected clips	Reduces CPU cost dramatically versus scanning every frame of the full source.

Render on demand	Avoids encoding 10-20 clips the user may never use.
No Kubernetes initially	A single host plus horizontally scalable queue workers is sufficient for MVP complexity and load.

27. Suggested Repository Structure

```

apps/
  web/          # Next.js
  api/          # NestJS
workers/
  media/        # Python BullMQ worker
  src/
    jobs/
      media/    # ffmpeg/ffprobe wrappers
      transcription/ # provider adapters
      analysis/  # candidate pipeline client/orchestration
      vision/    # face detection/tracking/reframe
      captions/
      storage/
packages/
  contracts/    # shared JSON schemas / OpenAPI-generated types where practical
infra/
  docker-compose.yml
  nginx-or-caddy/
  scripts/
docs/
  deployment.md
  pipeline.md

```

28. Implementation Notes for the Engineer

- Treat this document as the product contract; implementation details may vary when they preserve required behavior and interfaces.
- Prefer small, recoverable jobs over one giant process_video function.
- Do not store multi-gigabyte binary payloads in Redis/PostgreSQL.
- Do not perform heavyweight CPU work in the NestJS request lifecycle.
- Do not couple provider response formats directly to product entities.
- Use subprocess argument arrays and controlled working directories for FFmpeg and source download tools.
- Make render behavior deterministic from the persisted settings snapshot and pipeline version.
- Every stage should be observable and safely retryable before adding more features.
- Optimize for correctness and recoverability first; optimize encoding speed only after measuring real workloads.
- Capture pipelineVersion/model/provider metadata on results so future algorithm changes can be traced.

29. Appendix A - Reference State Machines

29.1 Video state machine

```

CREATED
-> UPLOADING (upload source only)

```

```
-> QUEUED
-> INGESTING
-> TRANSCRIBING
-> ANALYZING
-> READY
```

Any processing state -> FAILED

FAILED -> QUEUED/TRANSCRIBING/ANALYZING only through validated retry path

READY -> ANALYZING for an explicit re-analysis

29.2 Render state machine

QUEUED -> PREPARING -> ANALYZING_VISUALS -> RENDERING -> UPLOADING -> COMPLETED

Any active state -> FAILED

FAILED -> QUEUED through retry

COMPLETED is immutable; a settings change creates a new render/version

30. Appendix B - Example Queue Payloads

Ingest

```
{ "videoId": "vid_...", "pipelineVersion": "2026-09-mvp1" }
```

Transcription

```
{ "videoId": "vid_...", "transcriptVersion": 1, "provider": "hosted-default" }
```

Analysis

```
{ "videoId": "vid_...", "transcriptId": "tr_...", "analysisVersion": "mvp1" }
```

Render

```
{ "renderId": "rnd_...", "videoId": "vid_...", "settingsVersion": 1 }
```

31. Appendix C - Final Handoff Checklist

- OpenAPI or equivalent API specification generated and checked in.
- Database migrations checked in and reproducible.
- Docker Compose starts the complete local/staging stack.
- Environment variable template contains all required keys with descriptions and no secrets.
- Queue names, payload schemas and retry settings documented.
- At least one test transcription provider/mock allows local end-to-end development without paid calls.
- Golden media fixtures and render integration tests checked in or documented.
- R2 bucket CORS and lifecycle configuration documented.
- Production server setup, TLS, backups and deploy/restart procedure documented.
- PostgreSQL backup policy defined.
- Temp cleanup and disk alerting verified.
- Runbook contains common failure modes: provider outage, FFmpeg failure, Redis unavailable, R2 unavailable, low disk, stuck queue.
- MVP acceptance matrix signed off before P1 feature work begins.